

# MARS COLONY

## Proyecto programado I

Herencia y jerarquías de clases en Java

**Valor 20%**

# Descripción



• En el año 2187, una colonia humana en Marte depende de un núcleo central que produce oxígeno y energía para todos los módulos habitables.

• Una tormenta libera criaturas subterráneas que avanzan hacia la colonia. El jugador construye un perímetro y coloca unidades defensivas antes de cada ataque.

• Las unidades y las criaturas comparten propiedades, pero cada tipo ejecuta su ataque y movimiento de forma distinta.

• El objetivo académico es aplicar herencia, clases abstractas, polimorfismo y sobrescritura de métodos.

# 1 La expedición



• El juego es para una persona. Cada comandante usa un nombre único y puede mantener una partida guardada.

• La partida conserva nivel, capacidad del escuadrón, configuración del perímetro, unidades disponibles y progreso de cada componente.

• El juego inicia con 10 misiones. Al superar la décima, el jugador puede finalizar la campaña o generar nuevas misiones.



• Cada misión aumenta la resistencia y el daño de las criaturas. Las defensas también pueden mejorar.

• Al cargar una partida, el sistema debe reconstruirla exactamente en el estado en que se guardó.

# 2 Área de operación



## Mapa de la colonia

- Matriz gráfica de al menos 25 x 25 posiciones.
- El núcleo de oxígeno ocupa el centro del mapa.
- Las criaturas aparecen al azar fuera del perímetro construido.
- La interfaz distingue terreno libre, obstáculos, defensas, criaturas y núcleo.

## Reglas de ocupación

- Cada posición contiene como máximo un componente.
- Las unidades móviles cambian de posición durante la misión.
- El sistema valida límites y evita superposiciones.
- La pérdida del núcleo implica la derrota inmediata de la misión.

# 3 Escuadrón de defensa



- El comandante dispone de barreras y unidades defensivas con capacidades distintas.
- El escuadrón inicia con 20 puntos de capacidad. Cada misión superada agrega 5 puntos.
- La capacidad pertenece al escuadrón, no al mapa. Una defensa puede ocupar varios puntos aunque visualmente use una sola posición.
- Antes de iniciar la batalla, el jugador selecciona y coloca sus componentes sin exceder la capacidad disponible.
- Las defensas no pueden compartir posición con otro objeto.

# Características comunes



## Identidad y estado

- Identificador único y nombre.
- Imágenes para estado normal, movimiento y ataque.
- Vida inicial y vida actual.
- Nivel propio y misión de aparición.
- Posición en la matriz.

## Capacidad de combate

- Daño por golpe.
- Frecuencia de ataque por unidad de tiempo.
- Alcance y radio de efecto cuando corresponda.
- Costo de capacidad.
- Registro de objetivos atacados y atacantes recibidos.

# Tipos de defensa



- Contacto: solo ataca una criatura ubicada en una posición adyacente.
- Alcance medio: dispara a distancia dentro de su alcance configurado.
- Dron: se desplaza por el mapa, localiza un objetivo y ataca desde una posición válida.
- Impacto: detona cuando una criatura entra en su radio, causa daño de área y se destruye.
- Ataque múltiple: afecta varios objetivos o ejecuta varios disparos en un ciclo.
- Barrera: bloquea el paso y absorbe daño, pero no ataca.



# Configuración de componentes



- Un **programa independiente** permite administrar defensas y criaturas mediante usuario y contraseña.
- El administrador crea, modifica, consulta y desactiva configuraciones.
- Cada registro define el tipo concreto, atributos, imágenes y misión mínima de aparición.
- La información se almacena en archivos serializados o JSON. El juego carga únicamente componentes válidos desde esos archivos.
- La creación debe usar polimorfismo. El programa selecciona la subclase correcta según el tipo configurado.







# Ejemplos de defensas



| | | |
|---|---|---|
| **Pulso EMP** | **Impacto** | Vida 4 \| Daño 18 \| Radio 3 \| Capacidad 1 \| Misión 4 |
| **Torre láser** | **Alcance medio** | Vida 12 \| Daño 3 \| Alcance 6 \| Capacidad 2 \| Misión 1 |
| **Dron Centinela** | **Dron** | Vida 9 \| Daño 2 \| Alcance 1 \| Capacidad 3 \| Misión 3 |
| **Muro de titanio** | **Barrera** | Vida 30 \| Daño 0 \| Capacidad 1 \| Misión 1 |



Los valores son ejemplos. El administrador puede crear otras unidades dentro de las reglas del tipo.

# 4 Construcción del perímetro



- Antes de cada misión, el jugador coloca defensas y barreras en posiciones disponibles.
- El núcleo de oxígeno permanece fijo en el centro y no consume capacidad.
- El perímetro puede quedar abierto. La estrategia y las consecuencias pertenecen al jugador.
- Las defensas y las barreras bloquean el movimiento terrestre de las criaturas.
- La interfaz muestra capacidad total, capacidad utilizada y capacidad restante.
- Una batalla solo inicia si el mapa contiene el núcleo y cumple todas las reglas de colocación.



# El mapa



## Representación mínima esperada

**La presentación visual queda a criterio del grupo. Se evaluará que las posiciones, movimientos y ataques coincidan con el estado interno de los objetos.**

El mapa debe mostrar de forma clara:

- cuadrícula y coordenadas
- núcleo de oxígeno
- defensas y barreras
- criaturas terrestres y voladoras
- vida o estado de cada componente
- selección de una unidad para consultar su información



# 5 Batalla



- Las criaturas intentan destruir el núcleo. El jugador no controla ataques durante la batalla.
- Cada criatura busca un objetivo alcanzable, se desplaza cuando su tipo lo permite y ataca al cumplir la distancia requerida.
- Cada criatura y cada defensa activa se ejecuta en un hilo independiente.
- Una unidad puede atacar a varios objetivos y recibir ataques de varias unidades.
- La misión termina cuando el núcleo queda sin vida o cuando todas las criaturas han sido eliminadas.
- Después del resultado, el jugador puede repetir la misión o avanzar para facilitar la revisión del proyecto.



# Tipos de criaturas



## Terrestres

• Acechador: contacto y movimiento hacia el objetivo más cercano.

• Escupidor: ataque a distancia según su alcance.

• Demoledor: elige un objetivo, hace contacto, explota y causa daño en un radio.



## Especiales

• Volador: ignora barreras terrestres y solo recibe daño de unidades autorizadas para atacar objetivos aéreos.

• Enjambre: ejecuta ataques múltiples sobre uno o varios objetivos dentro del alcance.

• Cada tipo concreto redefine movimiento, selección de objetivo y ataque cuando corresponda.



# Jerarquía mínima sugerida



**ComponenteCombate**
**<<abstracta>>**

**Defensa**
**<<abstracta>>**

**Criatura**
**<<abstracta>>**

**Contacto | Alcance | Dron**
**Impacto | Múltiple | Barrera**

**Acechador | Escupidor | Volador**
**Demoledor | Enjambre**

La jerarquía es una guía. El grupo puede mejorarla, pero debe justificar las relaciones de herencia y evitar duplicar atributos o comportamiento común.

# Crecimiento por misión



• Al avanzar de misión, cada defensa y criatura aumenta de forma independiente entre 5% y 20% su vida máxima y su daño.

• El crecimiento se calcula sobre los valores de la misión anterior y debe quedar registrado.

• El sistema respeta la misión mínima de aparición de cada configuración.

• El ejército enemigo usa una capacidad total equivalente a la capacidad disponible del jugador para esa misión.

• Las nuevas misiones generadas después de la décima aplican las mismas reglas de crecimiento y selección.



# Persistencia



## Partidas

- Cantidad ilimitada de archivos.
- Nombre único de comandante.
- Nivel, capacidad y componentes.
- Posiciones y estado necesario para continuar.
- Opción para guardar, listar y cargar.

## Configuración

- Catálogo de defensas y criaturas.
- Tipo concreto y atributos.
- Rutas de imágenes.
- Estado activo o inactivo.
- Serialización o JSON con manejo de errores.

# Registro de combate



- Cada componente conserva su vida inicial, vida final, daño, frecuencia, tipo y posición final.
- Debe registrar cada objetivo atacado, cantidad de golpes y daño causado.
- También registra quién lo atacó, cantidad de golpes recibidos y daño recibido.
- Durante la batalla, al seleccionar un componente, la interfaz muestra su información y el historial acumulado hasta ese momento.
- Al terminar, una pantalla presenta todos los componentes participantes y permite revisar el detalle de cada uno.

# Requisitos obligatorios



## Diseño orientado a objetos

- Clases abstractas e interfaces cuando aporten al diseño.
- Herencia y sobrescritura de métodos.
- Colecciones polimórficas.
- Encapsulamiento y responsabilidades claras.
- Diagrama UML coherente con el código.

## Ejecución

- Interfaz gráfica para ambos programas.
- Matriz funcional de al menos 25 x 25.
- Hilos para unidades activas.
- Guardado y recuperación de archivos.
- Validaciones y manejo de errores.

# 6 Entrega y evaluación



## Entregables

- Programa de configuración.
- Programa del juego.
- Código fuente completo.
- **Diagrama de clases UML.**
- Archivos de ejemplo para probar.

## Condiciones

- Implementación en Java.
- Grupos de hasta dos personas.
- Entrega en TEC Digital.
- Fecha y hora: 13 de octubre 2026
- Valor: 20%.

**El diagrama de clases representa el 10% de la calificación del proyecto.**